Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Assignment No: 5

Date: 6th November 2026

Title: AMI, Manchester, B3ZS and 4B3T using python

Name: David Tapia Caicedo

Course: Bachelor of Engineering in Electronic Engineering

Subject: Digital Communications and Transmission

Lecturer: Ben Graham

Summary: To generate and investigate various line codes (AMI,Manchester,B3ZS and 4B3T) using python

OBJECTIVES:

After the completion of this experiment, students will have a good knowledge of the following line codes:

- Alternate Mark Inversion
- Manchester
- B3ZS
- 4B3T

EQUIPMENT REQUIRED:
Access to python on Kaggle.

THEORY:

AMI	A binary 1 is coded as alternate +V and –V for 50% of the bit interval and a binary 0 as 0V for the total bit interval.
Manchester	A binary 1 is coded as a +3 V to –3 V transition and a binary 0 as a –3 V to +3 V transition in the middle of the bit interval.
B3ZS	Bipolar with Three-Zero Substitution
PST	Pair Selected Ternary
4B3T	4 Binary 3 ternary block coding

Total Marks : 100

All work must be completed and submitted inside THIS Document

Part 1 – AMI – (Marks:10)

Question 1: What does AMI stand for? What is the fundamental operation of AMI, explain?

Is a variation of bipolar RZ format, during the bit interval, binary 0s are transmitted as no pulse. The operation of this method is while binary 1s, also called marks are transmitted as alternating positive and negative pulses. One binary 1 is transmitted as a positive pulse, while the next binary 1 is sent as a negative pulse. AMI is a blend of both level and transition codes, the binary 0 is a level code and the binary 1 acts like a transition code

- Using your python code from the previous lab 4, develop python code to generate AMI code for binary input 1011001.

The code will need to monitor the status of the previous mark

```
last_pulse = -A  # start alternating from +A
```

```
for i, bit in enumerate(data):
    if bit == '1':
        last_pulse = -last_pulse   # alternate
        signal[i*fs:(i+1)*fs] = last_pulse
```

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Initial parameters
bits = "1011001"  # Change to "01010101" for ASCII 'U'
A = 1             # Voltage amplitude
bit_duration = 1
fs = 100          # Samples per bit interval

# Signal initialization
total_duration = len(bits) * bit_duration
t = np.arange(0, total_duration, 1/fs)
signal = np.zeros_like(t)

last_pulse = -A   # Start alternating from +A

for i, bit in enumerate(bits):
    if bit == '1':
        last_pulse = -last_pulse  # Alternate polarity for each '1'
        signal[i*fs : (i+1)*fs] = last_pulse
    else:
        signal[i*fs : (i+1)*fs] = 0  # '0' stays at 0V

# Plot waveform
plt.figure(figsize=(10, 3))
plt.plot(t, signal, linewidth=2)
plt.ylim(-1.5, 1.5)
plt.xlabel("Time (s)")
plt.ylabel("Voltage (V)")
plt.title(f"AMI Line Coding for binary input {bits}")
plt.grid(True)
plt.show()
```

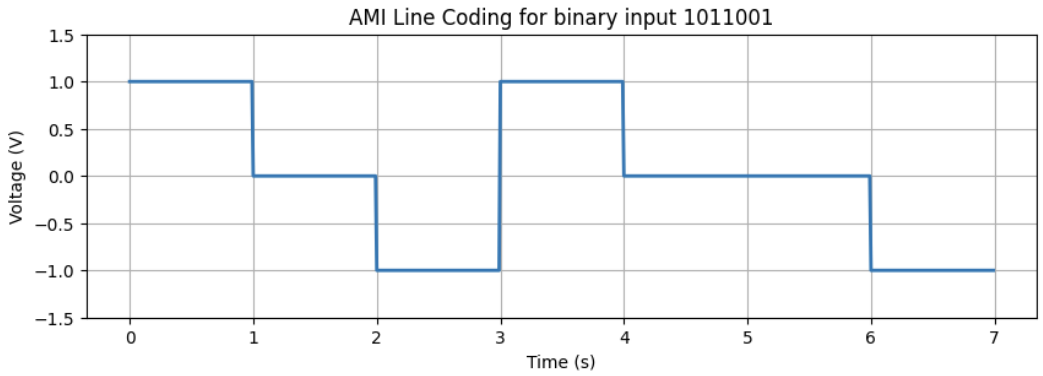


Figure 1: AMI Line Coding Waveform

Generate AMI output for ascii 'U'.

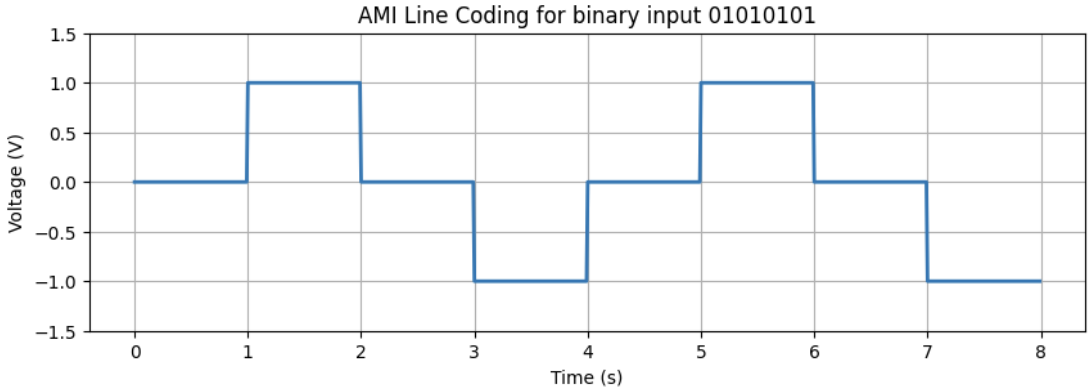


Figure 2: ASCII AMI Output for 'U'

The for loop goes through the bit string. If it finds a '1', it flips the sign of the last_pulse variable multiplying it by -1 and assigns that voltage level to the corresponding time interval of the signal. If it finds a '0', the signal stays at 0 V.

Part B – Manchester – (Marks:20)

Question 2: Explain the key features of Manchester coding

Manchester Coding is a Bi-Phase encoding and can be unipolar or bipolar. When a binary 1 is transmitted first as a positive pulse for one half of the bit interval and then as a negative for the remainder of the bit interval. And a binary 0 is transmitted first as a negative pulse for one half of the bit interval and then as a positive for the remainder of the bit interval.

Using your python code from the previous lab 4, develop python code to generate Manchester code for binary input 1011001 and for ASCII "!"

- Comment and explain key sections of the code

  Hint

```
    if bit == 1:
        # High then Low
        y_first = np.ones_like(t_first) * V_high
        y_second = np.ones_like(t_second) * V_low
    else:
        # Low then High
        y_first = np.ones_like(t_first) * V_low
        y_second = np.ones_like(t_second) * V_high

    # Append
    t.extend(t_first)
    t.extend(t_second)
    y.extend(y_first)
    y.extend(y_second)
```

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def plot_manchester(bits_str, title_label):
    V_high = 3
    V_low = -3
    bit_duration = 1

    t, y = [], []
    for i, b_char in enumerate(bits_str):
        bit = int(b_char)
        t_first = np.arange(i*bit_duration, i*bit_duration + bit_duration/2, 0.01)
        t_second = np.arange(i*bit_duration + bit_duration/2, (i+1)*bit_duration,
0.01)

        if bit == 1:
            # High then Low
            y_first = np.ones_like(t_first) * V_high
            y_second = np.ones_like(t_second) * V_low
        else:
            # Low then High
            y_first = np.ones_like(t_first) * V_low
            y_second = np.ones_like(t_second) * V_high

        # Append
        t.extend(t_first)
        t.extend(t_second)
        y.extend(y_first)
        y.extend(y_second)

    plt.figure(figsize=(10, 3))
    plt.plot(t, y, linewidth=2)
    plt.ylim(-4, 4)
    plt.xlabel("Time (s)")
    plt.ylabel("Voltage (V)")
```

```
    plt.title(f"Manchester Line Coding for {title_label}")
    plt.grid(True)
    plt.show()

# 1. Binary input 1011001
plot_manchester("1011001", "1011001")

# 2. ASCII '!' (Decimal 33 -> 8-bit binary 00100001)
ascii_excl = format(ord('!'), '08b')
plot_manchester(ascii_excl, f"'!' ({ascii_excl})")
```

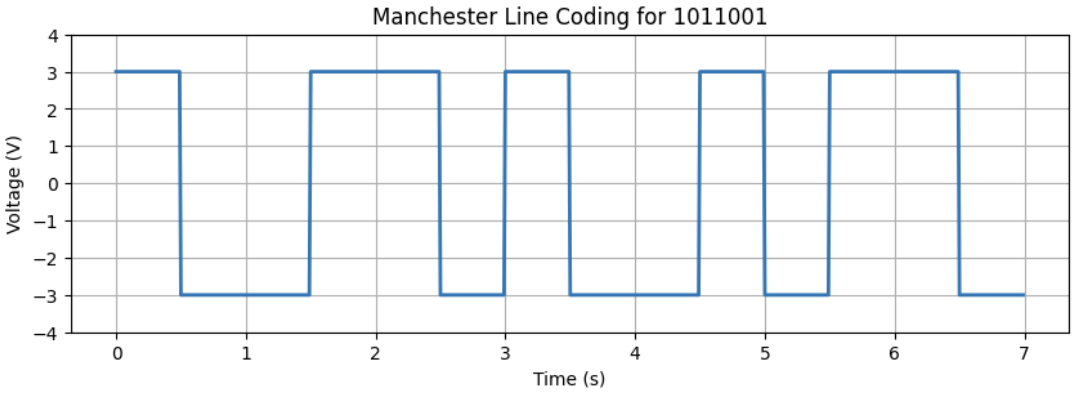


Figure 3: Manchester Line coding waveform for 1011001

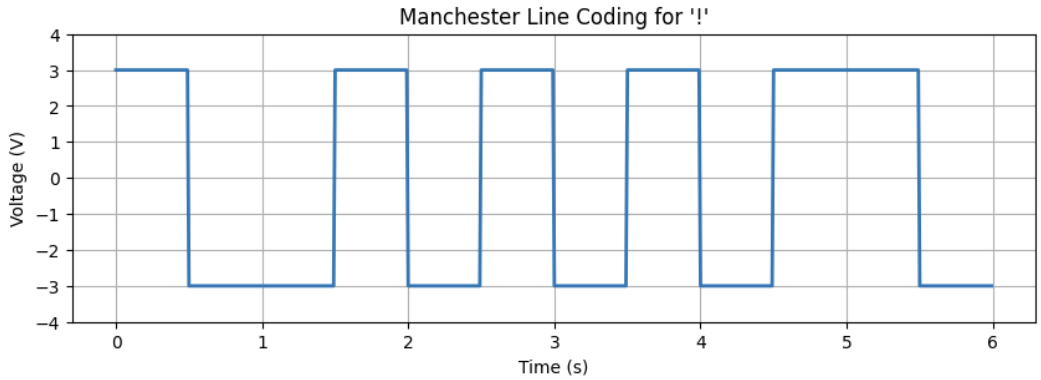


Figure 4: Manchester Line Coding Waveform for '!' (100001 in binary)

Key sections of the code

It's a biphase code where there are no voltage levels at 0 V to represent data unless there's no transmission. Each bit contains a mandatory transition in the middle of its interval, which ensures excellent clock recovery in the receiver. Where a bit 1 represents a transition from high to low, and a bit 0 represents a transition from low to high.

Part C – B3ZS – (Marks:20)

Question 3: Explain your understanding of B3ZS and what conditions need to be taken into account.

In a B3ZS code, a string of three consecutive zeros is replaced by B0V or 00V. B is a pulse that follows the AMI convention and V is a pulse violating the AMI convention. B0V or 00V is chosen such that the number of bipolar (B) pulses between the violations is odd. The conditions to take in account are the number of data bits 'B' that have passed since the last violation 'V', the polarity of the last marked pulse (+ or -) and depending on whether the number of marks since the last violation is even or odd, the 000 pattern is replaced by B0V or 00V applying the correct polarity according to the dash table._____

Number of B bits since last V	Pattern	Polarity of last B	Coded
Odd	00V	+	00+
		–	00-
Even	B0V	+	-0-
		–	+0+

On paper, create the B3ZS waveform for the binary input **1011000101000011**.

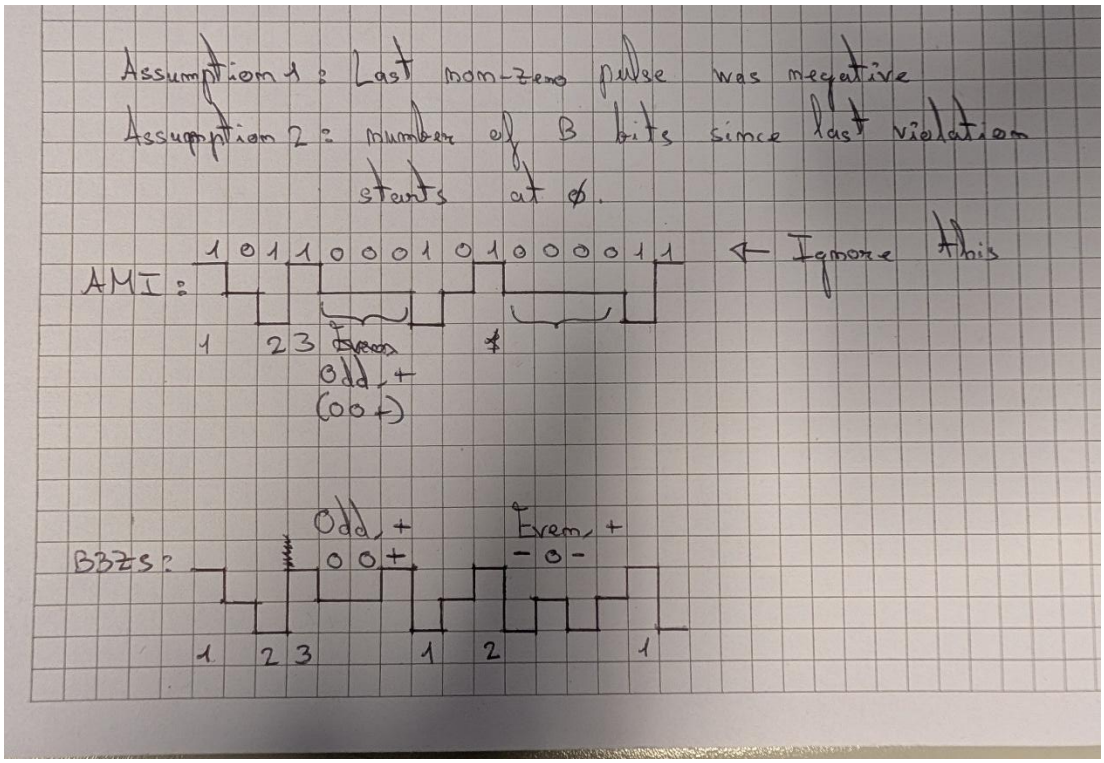


Figure 5: Calculate of 1011000101000011 in B3ZS encoding

Develop python code to generate B3ZS for binary input **1011000101000011**

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Input bitstream
bits = "1011000101000011"

# Parameters
V_high = 1
V_low = -1
bit_duration = 1
```

```
# Step 1: AMI encoding with B3ZS substitution
ternary_stream = []
last_polarity = V_low  # start assuming last non-zero was -V
(arbitrary choice)
marks_since_violation = 0
i = 0

while i < len(bits):
    if bits[i:i+3] == "000":  # found 3 zeros
        print("Three 0s detected")
        # Decide violation pattern based on state
        if last_polarity == V_high:
            if marks_since_violation % 2 == 0:  # Even number of
marks
                pattern = [V_low, 0, V_low]     # - 0 -
                print("Last Pulse was high and bits since violation
are even")
                last_polarity = V_low
            else:                               # Odd number of
marks
                pattern = [0, 0, V_high]        # 0 0 +
                print("Last Pulse was high and bits since violation
are odd")
                last_polarity = V_high
        else:
            if marks_since_violation % 2 == 0:  # Even number of
marks
                pattern = [V_high, 0, V_high]   # + 0 +
                print("Last Pulse was low and bits since violation
are even")
                last_polarity = V_high
            else:                               # Odd number of
marks
                pattern = [0, 0, V_low]         # 0 0 -
                print("Last Pulse was low and bits since violation
are odd")
                last_polarity = V_low

        ternary_stream.extend(pattern)
        marks_since_violation = 0  # reset after violation
        i += 3
    else:
        # Process individual bits ('1' or '0') using standard AMI
        if bits[i] == '1':
            last_polarity = -last_polarity
            ternary_stream.append(last_polarity)
            marks_since_violation += 1
        else:
            ternary_stream.append(0)
```

```
        i += 1

print("Input bits:   ", bits)
print("B3ZS encoded: ", ternary_stream)

# Step 2: Generate waveform
t = []
y = []

for i, sym in enumerate(ternary_stream):
    t_sym = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
    y_sym = np.ones_like(t_sym) * sym
    t.extend(t_sym)
    y.extend(y_sym)

# Step 3: Plot waveform
plt.figure(figsize=(12, 4))
plt.plot(t, y, linewidth=2)
plt.ylim(-V_high-1, V_high+1)
plt.yticks([V_low, 0, V_high])
plt.xlabel("Time")
plt.ylabel("Voltage")
plt.title("B3ZS Line Coding - 1011000101000011")
plt.grid(True)
plt.show()
```

- Walk through the critical parts of your Python script. Outline and explain the core functionality of your Python code.

  The variables last_polarity and marks_since_violation are used to keep track of the polarity of the last transmitted pulse and the number of binary ones sent since the previous violation. The algorithm processes the input sequence using a sliding window to detect whether the next three bits are equal to 000. When this pattern is found, it checks the parity of the number of marks transmitted and the polarity of the previous pulse to determine which of the four possible replacement sequences should be generated: [-1, 0, -1], [0, 0, 1], [1, 0, 1] or [0, 0, -1]. Once the appropriate sequence has been selected, the counter is reset to zero and the index moves forward three positions. If three consecutive zeros are not detected, the algorithm follows the standard AMI encoding procedure, whenever a 1 is encountered, the polarity is inverted and the mark counter is increased by one, whereas a 0 is represented by 0 volts. In this case, the index advances only one position before processing the next bit.
- Include your generated waveform here. Compare with the waveform you expected.

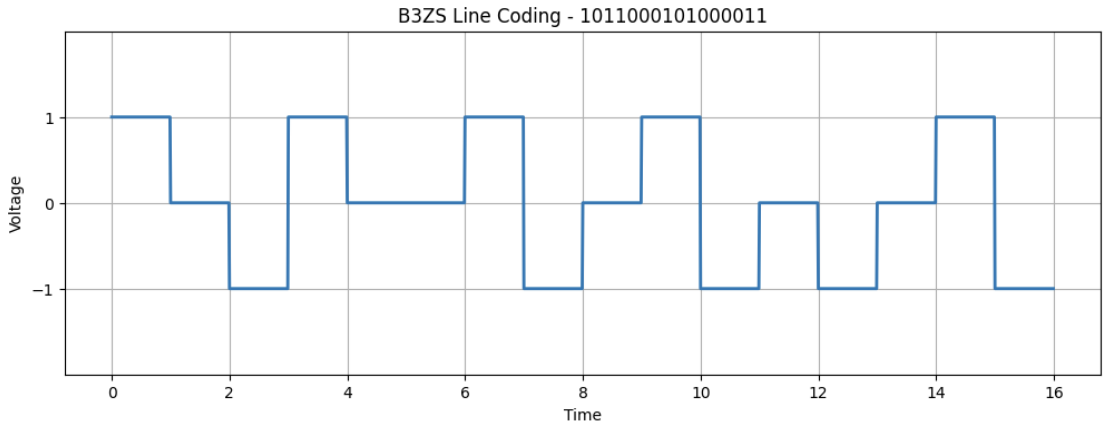


Figure 6: B3ZS Line Coding

Repeat for 0x 2C28 (Hex). Assume previous mark was postive (+) and an odd number of marks since last violation.

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Convert 0x2C28 to 16-bit binary: '0010110000101000'
bits = format(0x2C28, '016b')

# Parameters
V_high = 1
V_low = -1
bit_duration = 1

# Step 1: AMI encoding with B3ZS substitution
ternary_stream = []
last_polarity = V_high       # Assume previous mark was positive (+)
marks_since_violation = 1    # Assume an odd number of marks since last violation
i = 0

while i < len(bits):
    if bits[i:i+3] == "000":  # found 3 zeros
        print("Three 0s detected")
        # Decide violation pattern based on state
        if last_polarity == V_high:
            if marks_since_violation % 2 == 0:  # Even number of marks
                pattern = [V_low, 0, V_low]     # - 0 -
                print("Last Pulse was high and bits since violation are even")
                last_polarity = V_low
            else:                               # Odd number of marks
                pattern = [0, 0, V_high]        # 0 0 +
                print("Last Pulse was high and bits since violation are odd")
                last_polarity = V_high
        else:
            if marks_since_violation % 2 == 0:  # Even number of marks
                pattern = [V_high, 0, V_high]   # + 0 +
                print("Last Pulse was low and bits since violation are even")
                last_polarity = V_high
            else:                               # Odd number of marks
                pattern = [0, 0, V_low]         # 0 0 -
                print("Last Pulse was low and bits since violation are odd")
                last_polarity = V_low
```

```
            ternary_stream.extend(pattern)
            marks_since_violation = 0  # reset after violation
            i += 3
        else:
            # Process individual bits ('1' or '0') using standard AMI
            if bits[i] == '1':
                last_polarity = -last_polarity
                ternary_stream.append(last_polarity)
                marks_since_violation += 1
            else:
                ternary_stream.append(0)
            i += 1

print("Input bits:   ", bits)
print("B3ZS encoded: ", ternary_stream)

# Step 2: Generate waveform
t = []
y = []

for i, sym in enumerate(ternary_stream):
    t_sym = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
    y_sym = np.ones_like(t_sym) * sym
    t.extend(t_sym)
    y.extend(y_sym)

# Step 3: Plot waveform
plt.figure(figsize=(12, 4))
plt.plot(t, y, linewidth=2)
plt.ylim(-V_high-1, V_high+1)
plt.yticks([V_low, 0, V_high])
plt.xlabel("Time")
plt.ylabel("Voltage")
plt.title(f"B3ZS Line Coding - 0x2C28 ({bits})")
plt.grid(True)
plt.show()
```

- Include your generated waveform here. Discuss output

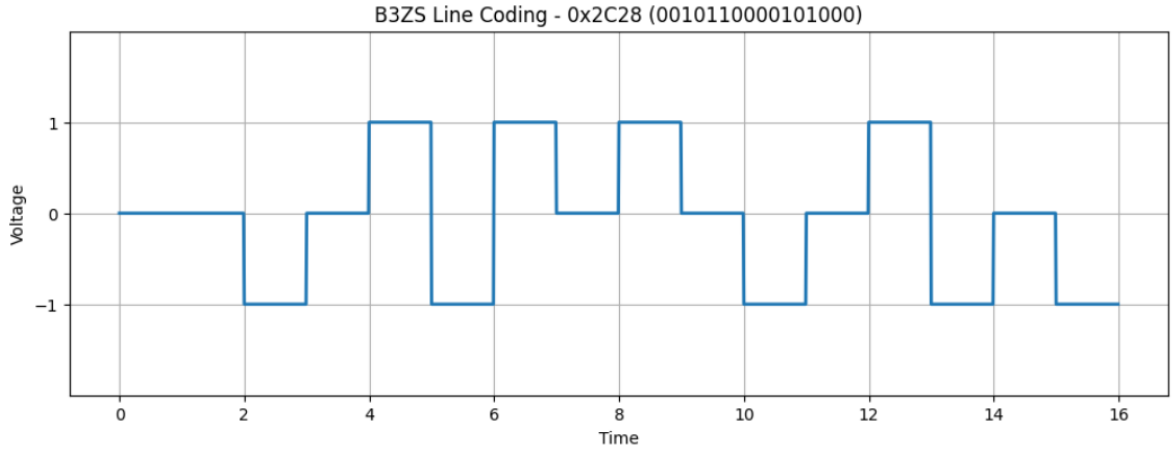


Figure 7: B3ZS Line encoding for ***0010110000101000***

Hint:

Remember to generate the wave form:

```
# Step 2: Generate waveform
t = []
y = []

for i, sym in enumerate(ternary_stream):
    t_sym = np.arange(i*bit_duration, (i+1)*bit_duration, 0.01)
    y_sym = np.ones_like(t_sym) * sym
    t.extend(t_sym)
    y.extend(y_sym)
```

Part D-PST – (Marks:20)

Question 4: Explain your understanding of the operation of PST encoding

PST is a block-coding scheme that groups binary data into pairs of 2 bits (00, 01, 10, 11) and translates each pair into a 2-symbol ternary codeword using +, -, and 0 levels. To prevent DC component build-up, the encoder operates in two alternating modes: + Mode and - Mode. Pairs with balanced pulses have zero DC weight and do not switch modes. However, pairs with a single non-zero pulse, 10 and 01, carry a net charge, so they use the current mode's polarity and trigger a mode switch, from + Mode to - Mode or vice versa, for the next pair, ensuring that every net pulse is balanced by the next one.

- Solve on paper for in 0xB52E (Hex). Assume – Mode to start. Include an image of the output waveform and decisions taken

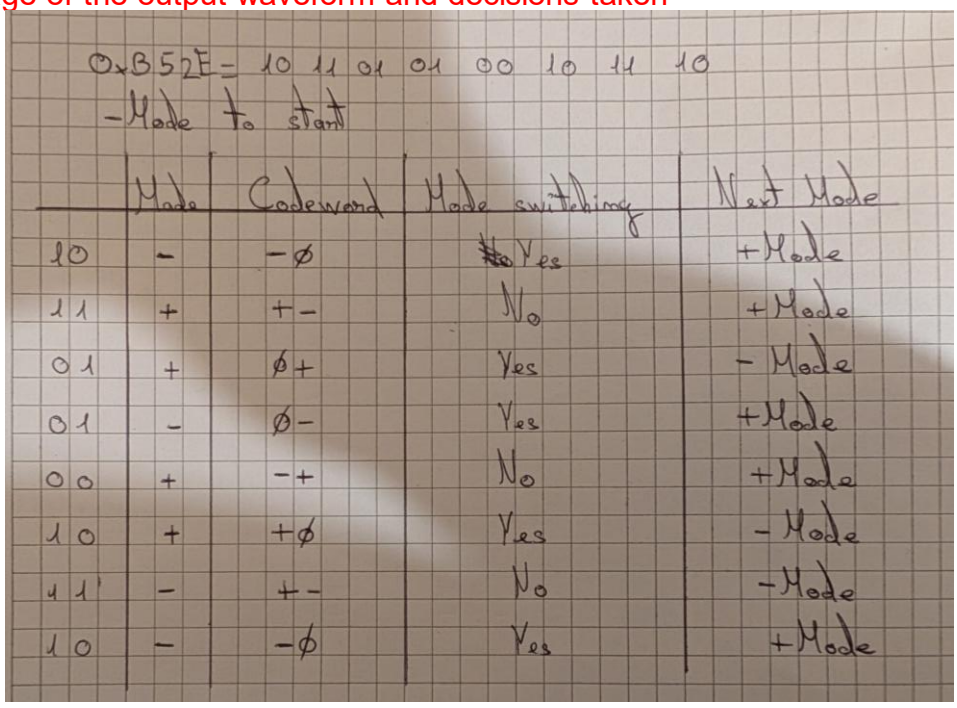


Figure 8: Calculate of 0xB52E in PST encoding

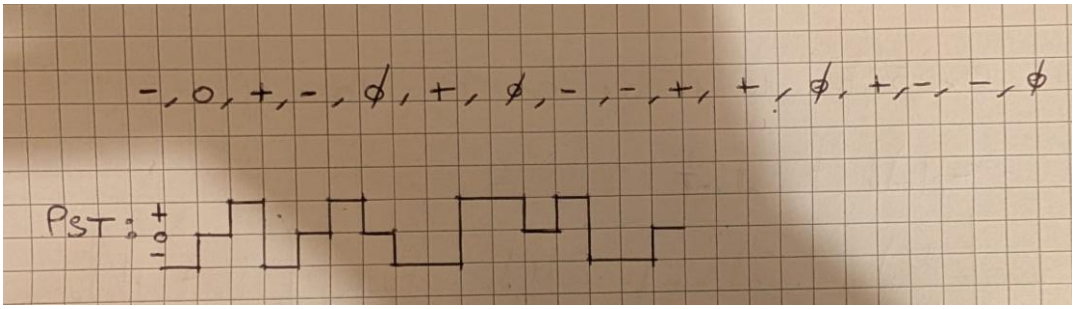


Figure 9: PST code and waveform

Binary Pair	Ternary Codewords + Mode	Ternary Codewords – Mode	Mode Switching
11	+ –	+ –	No
10	+ 0	– 0	Yes
01	0 +	0 –	Yes
00	– +	– +	No

Part E-4B3T – (Marks:20)

Question 5: Explain your understanding of 4B3T, what conditions need to be taken into account and the steps that have to be taken. Refer to Notes

In 4B3T coding, words consisting of 4 binary digits are mapped into three ternary symbols. The encode monitors the integer variable I = Np – Nn, where Np is the number of positive pulses and Nn are the number of negative pulses transmitted. So, keeping track on the DC content/balance.
There are 3 types of codewords, column 1 = negative dc, column 2 = zero dc and column 3 = positive dc.
The main aim is to maintain a zero-dc level, by selecting most appropriate ternary codewords.
The conditions to take in count are If I > 0, choose codeword from columns 1 & 2, If I < 0, choose codeword from columns 2 & 3, If I = 0, choose codeword from columns 2, and from 1, if the previous I > 0 and If I = 0, choose codeword from columns 2, and from 3, if the previous I < 0.
The steps that have to be taken are: divide the data into 4 bits, state your assumed previous bit, select first tenary codeword and determine the sequence of pulses transmitted for binary sequence below if ternary code 4B3T is used. This time the previous state was negative pulse

---

- Develop the 4B3T coding and output waveform on paper for binary input 10110001010000011010111101. Include images of your working out in the report. State any assumptions. Note : I = 1

Binary Words	Ternary Codewords		
	Column 1	Column 2	Column 3
0000	- - -		+ + +
0001	- - 0		+ + 0
0010	- 0 -		+ 0 +
0011	0 - -		0 + +
0100	- - +		+ + -
0101	- + -		+ - +
0110	+ - -		- + +
0111	- 0 0		+ 0 0
1000	0 - 0		0 + 0
1001	0 0 -		0 0 +
1010		0 + -	
1011		0 - +	
1100		+ 0 -	
1101		- 0 +	
1110		+ - 0	
1111		- + 0	

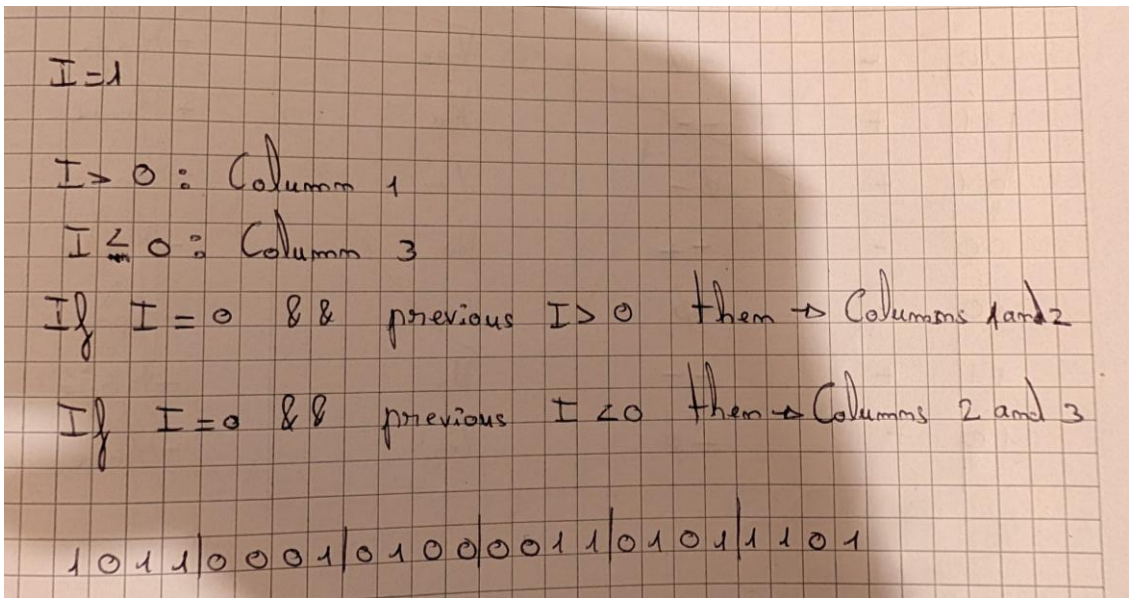


Figure 10: Before calculating of 4B3T encoding

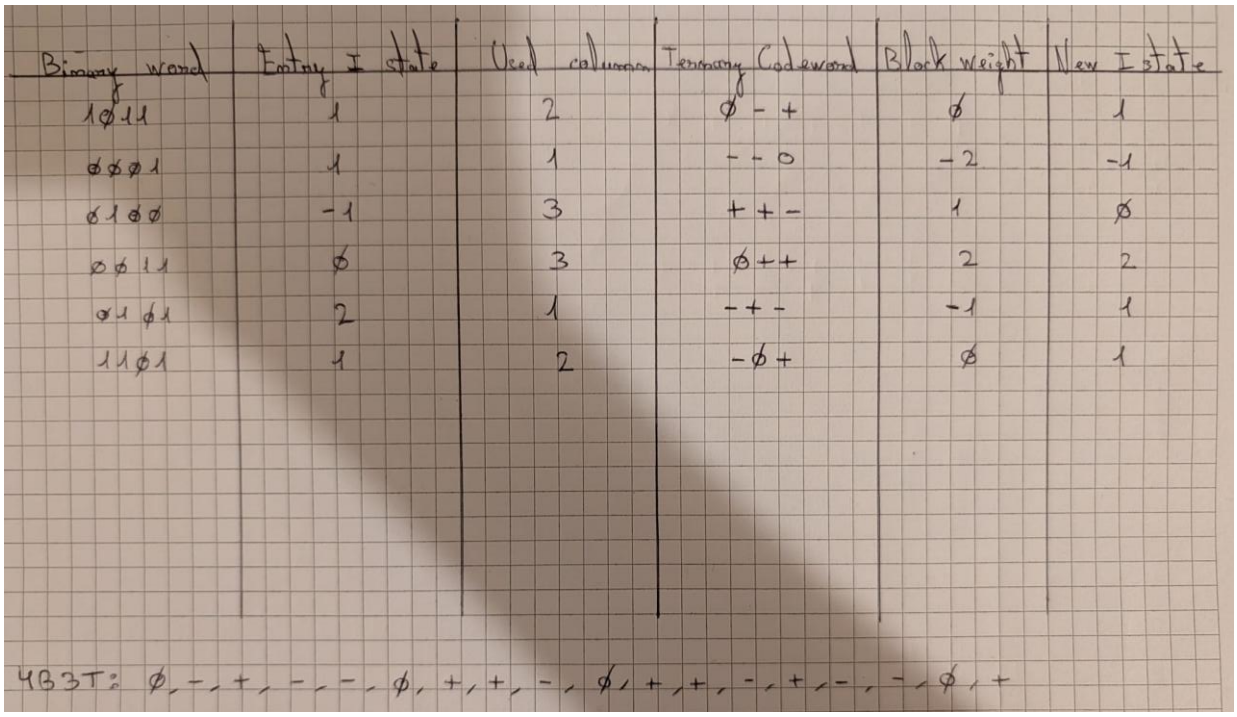


Figure 11: Calculate of 4B3T encoding

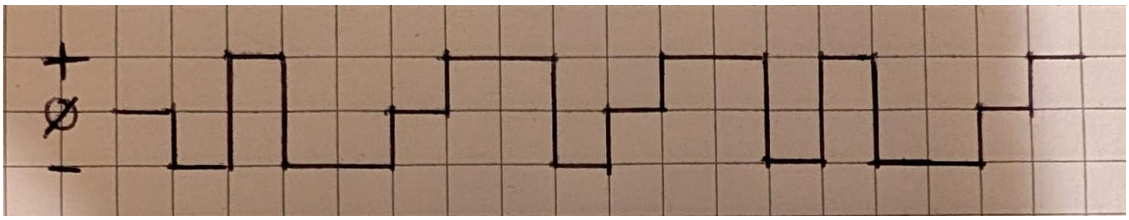


Figure 12:4B3T waveform

- Using the python code attached, run and confirm if it is correct.

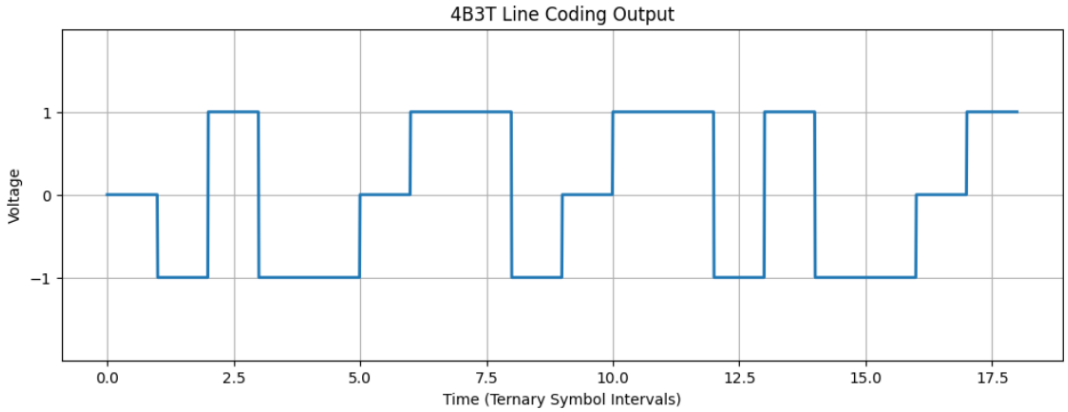


Figure 13: 4B3T Line Coding Waveform in Python code

Code:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# 4B3T Lookup Tables (Column 1: negative DC, Column 2: zero
DC, Column 3: positive DC)
col1 = {
    "0000": [-1, -1, -1], "0001": [-1, -1,  0], "0010": [-1,
0, -1], "0011": [ 0, -1, -1],
    "0100": [-1, -1,  1], "0101": [-1,  1, -1], "0110": [ 1, -
1, -1], "0111": [-1,  0,  0],
    "1000": [ 0, -1,  0], "1001": [ 0,  0, -1]
}
col2 = {
    "1010": [ 0,  1, -1], "1011": [ 0, -1,  1], "1100": [ 1,
0, -1],
    "1101": [-1,  0,  1], "1110": [ 1, -1,  0], "1111": [-1,
1,  0]
}
col3 = {
    "0000": [ 1,  1,  1], "0001": [ 1,  1,  0], "0010": [ 1,
0,  1], "0011": [ 0,  1,  1],
    "0100": [ 1,  1, -1], "0101": [ 1, -1,  1], "0110": [-1,
1,  1], "0111": [ 1,  0,  0],
    "1000": [ 0,  1,  0], "1001": [ 0,  0,  1]
}

bits = "101100010100001101011101"
I = 1  # Initial running digital sum (Note: I = 1)
ternary_stream = []

for idx in range(0, len(bits), 4):
    word = bits[idx:idx+4]
    if word in col2:
        code = col2[word]
        col_used = "Column 2"
    elif I > 0:
        code = col1[word]
        col_used = "Column 1"
    else:
        code = col3[word]
        col_used = "Column 3"

    weight = sum(code)
    print(f"Word: {word} | State I (in): {I:2d} | {col_used} -
> {code} | State I (out): {I + weight:2d}")
    I += weight
    ternary_stream.extend(code)

# Generate and plot waveform
bit_duration = 1
t, y = [], []
for i, sym in enumerate(ternary_stream):
```

```
    t_sym = np.arange(i*bit_duration, (i+1)*bit_duration,
0.01)
    y_sym = np.ones_like(t_sym) * sym
    t.extend(t_sym)
    y.extend(y_sym)

plt.figure(figsize=(12, 4))
plt.plot(t, y, linewidth=2)
plt.ylim(-2, 2)
plt.yticks([-1, 0, 1])
plt.xlabel("Time (Ternary Symbol Intervals)")
plt.ylabel("Voltage")
plt.title("4B3T Line Coding Output")
plt.grid(True)
plt.show()
```

- Highlight and discuss the various sections of the code.

The script performs 4B3T encoding by using three lookup dictionaries, named col1, col2 and col3, which contain the ternary codewords associated with negative, neutral and positive DC weights. The input bit sequence, consisting of 24 bits, is processed in groups of four through a for loop, while the variable I, initially set to 1, keeps track of the cumulative digital sum. When a binary group corresponds to one of the last six entries in the encoding table, from 1010 to 1111, the encoder directly uses col2, as these combinations are already DC-balanced. For the other possible groups, from 0000 to 1001, the selection depends on the current value of I: if it is greater than zero, a codeword from col1 is chosen to compensate for the positive accumulated weight; otherwise, the encoder selects a codeword from col3. Once each four-bit block has been converted into its corresponding ternary symbols, the value of I is updated by adding the sum of the selected codeword. Finally, comparing Figure 12 with the manually completed encoding table in Figure 11 shows that both methods generate the same sequence of 18 ternary symbols: 0, -, +, -, -, 0, +, +, -, 0, +, +, -, +, -, -, 0, +.

Part E – Conclusion – (Marks:10)

Explain the main characterises of each coding scheme, the steps taken when creating them and any advantages and disadvantages of each.

In this lab, we generated and tested five-line coding schemes to see how they deal with DC balance, bandwidth, and clock synchronisation. AMI is the simplest bipolar scheme, created by keeping binary 0s at 0 V and flipping the polarity between +V and -V on every consecutive binary 1. This gives AMI a zero DC component and basic error detection through bipolar violations, though long runs of zeros leave the signal flat at 0 V and cause the receiver to lose clock synchronisation.

Manchester coding solves this timing issue by splitting each bit period in half so that a binary 1 transitions from +3 V to -3 V and a binary 0 transitions from -3 V to +3 V right in the middle of the bit. Because there is always a transition in the middle of the bit, clock recovery is much easier and there is no DC component, although the downside is these constant transitions require twice the transmission bandwidth of NRZ codes.

B3ZS improves on standard AMI without doubling the bandwidth by tracking both the last pulse polarity and the number of marks sent since the last violation. Whenever three consecutive zeros occur, the encoder substitutes them with 00V, if the mark count is odd, or B0V, if even. This guarantees frequent transitions for the receiver clock even during long zero strings while keeping the DC level balanced, with the only disadvantage being the extra hardware logic needed to track states and detect deliberate violations.

Finally, block codes like PST and 4B3T group bits together before mapping them into ternary levels (+, -, 0). PST pairs binary bits into 2-bit blocks and switches between + Mode and - Mode whenever an unbalanced pair (01 or 10) is sent, preventing DC wander while keeping the symbol rate unchanged. 4B3T goes a step further by mapping 4 binary bits into just 3 ternary symbols using a three column table controlled by the running digital sum I = Np - Nn. By sending 4 bits in 3 symbol intervals, 4B3T reduces the required baud rate and bandwidth by 25% while maintaining strict DC balance, though it requires the most complex framing and lookup logic at both ends of the link.